Python Virtual Environments

Magnus Hagdorn

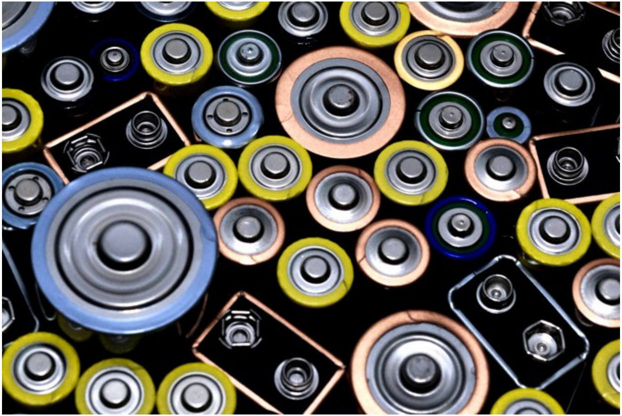
School of GeoSciences
University of Edinburgh

June 21, 2018

THE UNIVERSITY of EDINBURGH

Python Virtual Environments
Outline

Python Modules
Batteries included



CC Lucius Catalan

Python Modules

Python Modules and Packages

- python modules are python files containing executable statements and function/class definitions
- python packages group modules together in a directory which also contains a `__init__.py` module
- use them with
  import some_module
- find lots of useful packages from the cheese shop at

  https://pypi.org/

Python Modules
Python Module Search Path

Python modules are searched for in

- the directory containing the script or the current directory
- `PYTHONPATH`, a list of directories (separated by : on Linux/Mac or ; on Windows)
- the installation-dependent default

in a python shell you can check:

```
import sys
print(sys.path)
```

Python Modules

Getting Python Modules

- on Linux - use your distribution package manager such as yum, apt (or speak to your sys-admin to install it for you)
- write your own module and copy it somewhere
- download a module and install iy by hand
  `python ./setup install`
- use
  `pip install some-package`

Python Modules
Installation Location

both `setup.py` and `pip` will

- install into the installation default (you might need admin access)
- `--user`:install into a user writeable location
- `--prefix /some/dir`: specify directory into which to install and set `PYTHONPATH`

Python Environments

CC Matthew Paulson

Python Environments

- provide isolated environment into which you can install python modules
- you can have multiple enviornments with different versions

Python Environments
virtualenv

1. install virtualenv using your distribution package manager or `pip`
2. create a virtual environment
   `virtualenv /some/directory`
3. or if you want access to existing package
   `virtualenv /some/directory --system-site-packages`
4. activate the virtual environment
   `source /some/directory/bin/activate`
5. when done run
   `deactivate`

Python Environments
virtualenvwrapper

makes virtualenv more convinient

1. install virtualenvwrapper using your distribution package manager or `pip` (on windows install virtualenvwrapper-powershell)
2. setup a directory for your virtual environments
3. create a virtual environment
   `mkvirtualenv venv`
4. use the virtual environment
   `workon venv`
5. when done
   `deactivate`
6. and finally, remove it
   `rmvirtualenv venv`

Python Environments
virtualenvwrapper

on Linux

```
export WORKON_HOME=~/Envs
# if you installed it using pip you
# need to source the wrapper
source /usr/local/bin/virtualenvwrapper.sh
# make sure the environment directory is created
mkdir $WORKON_HOME
```

on Windows

using powershell

```
$env:WORKON_HOME="~/Envs"
mkdir $env:WORKON_HOME
import-module virtualenvwrapper
```

requirements files record python packages and their versions:

```
package_a
package_b == 0.10.2
package_c > 0.2
```

- install packages from requirements file
  `pip install -r requirements.txt`
- generate requirements file from currently installed packages
  `pip freeze > requirements.txt`

Python Environments
Anaconda

- a python (2 and 3) distribution
- available for Linux, Windows and Macs
- not only python packages but also other libraries such as gdal, etc
- full distribution or bare miniconda

https://www.anaconda.com/

Python Environments
Anaconda

- create a conda environment
  `conda create -n venv`
- or with a specific path
  `conda create -p /some/dir/venv`
- activate environment
  `source activate venv`
- and finally
  `source dactivate`

Python Environments
Anaconda

install packages

- using `pip`
- or

  `conda install package`
- multiple channels such as conda-forge:

  https://conda-forge.org/

  `conda config --add channels new_channel`

Warning

It's easy to get conflicts between packages from different channels and/or the base installation. You can avoid these by installing packages from source using `pip`

- lots of ways of managing conda environments
- similar to `requirements.txt` file you can use an `evironment.yml` file:

```
name:  myenv
channels:
 - conda-forge
dependencies:
 - python=2.7
 - pandas
 - pip:
   - geopandas
```

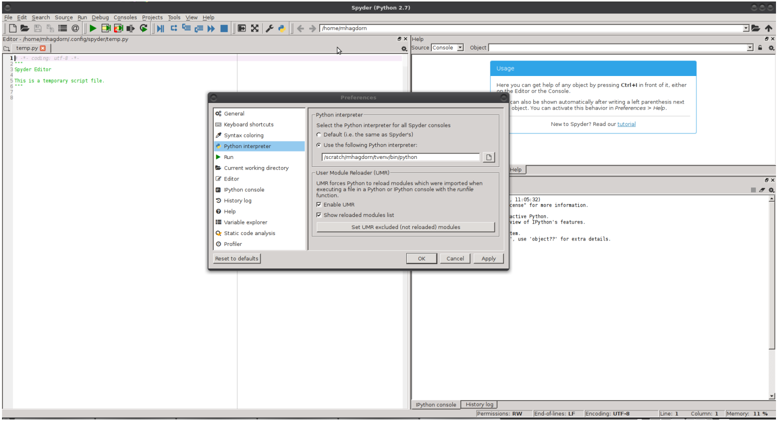
Python Environments
IDEs and Virtual Environments

install IDE into virtual env

- `pip install spyder`
- `conda install spyder`
- in newer versions of spyder you can change the interpreter

Python Environments

IDEs and Virtual Environments



Conclusion

CC Magnus Hagdorn

Recommendations

1. (on Linux) use system packages (or ask your sys-admins)
2. on Windows/Mac use standard conda
3. if you require particular versions use virtual environments
   1. use conda environments if you also need support libraries such as gdal
   2. otherwise use virtualenv/virtualenvwrapper
4. virtual environments are also very useful when you need different versions of your software (production and development version)